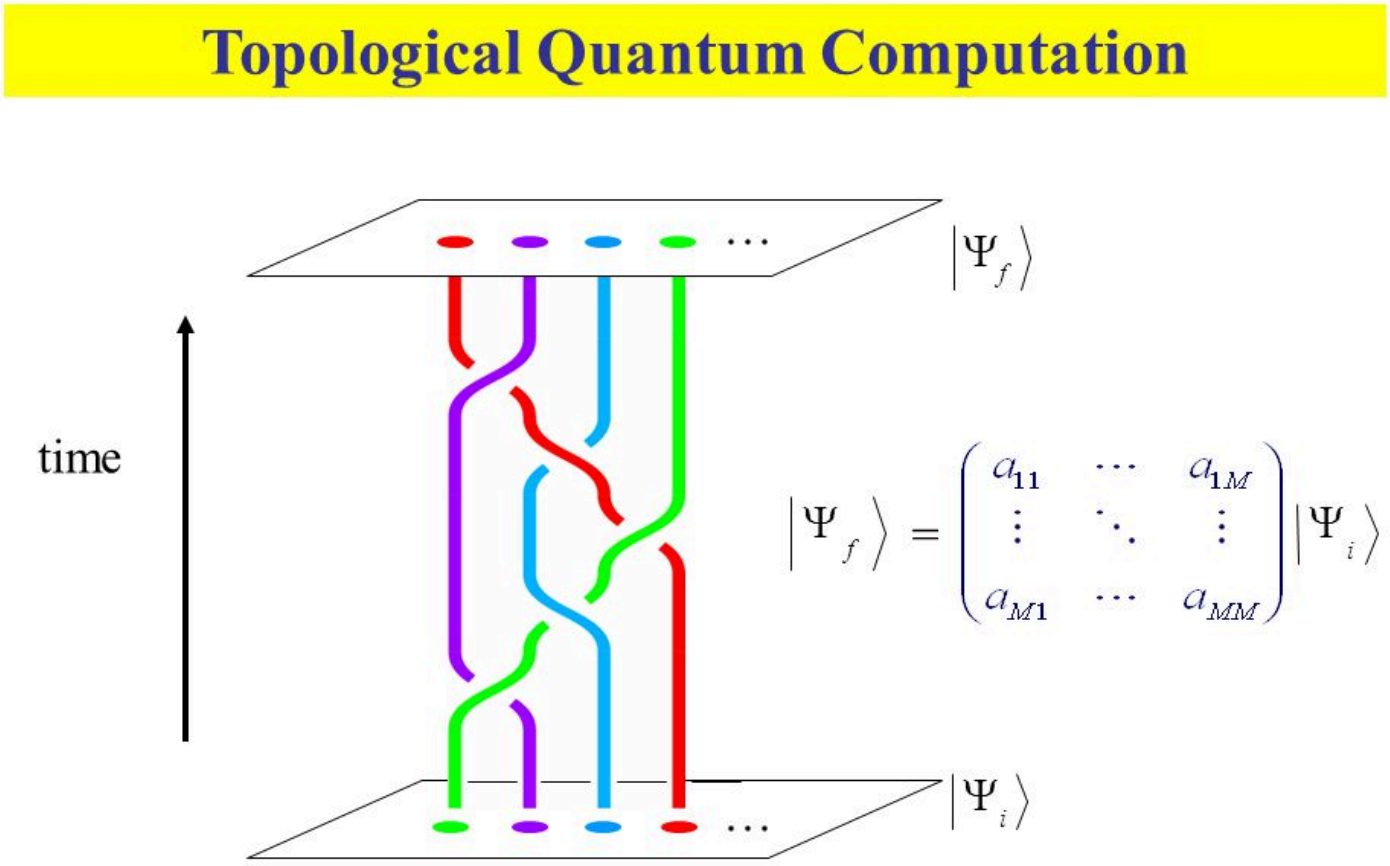
Fibonacci任意子在拓扑量子计算中的作用

选题背景

选择研究这个模型原因相当直接，一来是之前看到微软宣布搞拓扑量子计算，感觉非常新颖。另一方面是最近科研训练和这一部分有些相关，于是决定研究一下。

Fibonacci 任意子模型理论

拓扑量子计算利用二维的任意子，利用辨群结构充当逻辑门来进行计算。



模型前置要求：

其中任意子主要用到的是Non Abelian Anyons： 即不同交换顺序不一定可交换;

然后需要融合代数：

$a \times b = \sum_c N_{bc}^a \, c$（$N_{bc}^a$为非负整数）

辫子表示：给定 n个任何子，辫子群 B_n在结合空间 H上有表示 $\rho : Bn \to U(H)$；每个生成元 σi（交换第 i 与 i+1粒子）映为幺正矩阵 $\rho(\sigma i)$。

计算 σi的常用步骤：若在当前 fusion tree 下直接对角，则 $\sigma i = diag(R)$；否则用 F变基再施加 diag(R)，再变回原基：$\sigma i = F - 1 \, diag(R)$。 其中：

- F是多粒子结合顺序变换的矩阵：决定不同融合顺序下态的展开系数
- R给出交换两个任何子的局部相位或矩阵元：决定把两个任何子绕动一圈的局部效果

而模型Fibonacci anyon: {1,τ}，满足的融合规则是$\tau \times \tau = 1 + \tau$，其辫子表示在逻辑子空间上是稠密的，单靠编辫子即可近似任意单比特幺正（即拓扑上普适）。其中任意子的量子维数$d_\tau = \frac{1+\sqrt{5}}{2}$

拓扑量子比特的自然编码：以 **三个 τ 任意子** 编码一个 qubit ： $V_1 = \phi = span\{|0_L > |1_L >\}$

模型的一些工具：

F-matrix（结合变换矩阵）

Fibonacci 流形只有一个非平凡的粒子 τ，因此 F 矩阵只有一个非平凡块：

$$F_\tau^{\tau\tau\tau} = \begin{pmatrix} \phi^{-1} & \phi^{-1/2} \\ \phi^{-1/2} & \phi^{-1} \end{pmatrix}$$

R-matrix（交换矩阵 / 编织矩阵）

Fibonacci 任意子的交换相位为：

$R_1^{\tau\tau} = e^{-4\pi i/5}$， $R_\tau^{\tau\tau} = e^{3\pi i/5}$, R 矩阵操作的是两个 τ 任意子的融合通道。

对于三任意子，我们关心两个基本 braid generators: σ_1, σ_2:

其中：

$$\sigma_1 = \begin{pmatrix} e^{-4\pi i/5} & 0 \\ 0 & e^{3\pi i/5} \end{pmatrix}, \sigma_2 = F^{-1}\sigma_1 F$$

然后可以考虑构建量子门：

首先是拓扑量子比特的逻辑基：三 τ 任意子编码一个 qubit：

- |0⟩：前三个 τ 融合为 1
- |1⟩：前三个 τ 融合为 τ

Hilbert 空间维数 = 2，即所有 braid sequences 最终都是作用在 2×2 的 SU(2) 矩阵上。

用 braid 来实现量子门（核心）

使用 σ_1, σ_2 的某些序列，可以逼近任意单比特门。

Fibonacci 任意子实现 NOT 门: $X \approx (\sigma_1\sigma_2^{-1}\sigma 1)^5$ ， Hadamard 门: $H \approx \sigma_1\sigma_2\sigma_1\sigma_2^2\sigma 1$

模型的一般方法

1. 构造 F, R 矩阵

2. 构建 braid generators σ_1, σ_2

3. 编译 braid sequences → SU(2) 矩阵

比较结果与目标量子门的 Frobenius 距离

$$\epsilon = \|U_{\text{braid}} - U_{\text{target}}\|$$

4. 使用搜索算法找到最短 braid word

搜索常用方法可以用：BFS（长度优先搜索），A* 搜索 , Solovay–Kitaev 递归

以上是做 Fibonacci 任意子 TQC 的常规方法。

解决问题

这里将尝试使用以上方法，把经典的逻辑门，考虑用拓扑量子计算的想法实现。基于这些便能够实现拓扑量子计算。

详细代码可见: fibonacci_braid.py 以及 demo_run.py

这里选取几个核心来解释一下：

1. F 和 R 的物理定义

```
phi = (1 + 5**0.5) / 2
phi_inv = 1 / phi

# F-matrix for tau,tau,tau -> tau block (2x2)
F = np.array([[phi_inv, phi**(-0.5)],
              [phi**(-0.5), -phi_inv]], dtype=complex)

# R diag: R^{tau tau}_1 and R^{tau tau}_tau
R1 = np.exp(-4j * np.pi / 5)
Rtau = np.exp(3j * np.pi / 5)
Rdiag = np.diag([R1, Rtau])
```

- F 矩阵（3 任意子重关联变换）是整个逻辑比特空间的基底变换
- R 是任意子交换（braiding）的基本相位

这两个矩阵决定了物理：

- F 和 R 是拓扑量子计算的 核心数据

- 整个 braid group 的 2×2 表示都由它们生成

2. 比较两个量子门：

```
tr = np.trace(V.conj().T @ U)
phi_opt = np.angle(tr)
diff = U - np.exp(1j * phi_opt) * V
return np.linalg.norm(diff, ord='fro')
```

量子力学里全局相位不可观察： U 和 e^{iφ} U 是同一个门

所以比较两个门时要把相位优化掉：

$$\phi_{\text{opt}} = \arg(\text{Tr}(V^\dagger U))$$

这是最优全局相位，使 Frobenius 距离最小。

这段代码就是计算一个 braid 实现的门 U 与目标 V 的可观察误差

3.BFS搜索算法实现：

```
q = deque([[]])   # start with empty word

while q:
    w = q.popleft()
    U = braid_word_to_matrix(w)

    # pruning key
    key = tuple(np.round(np.concatenate([U.real.flatten(), U.imag.flatten()]), 12))
    if key in seen:
        pass
    else:
        seen[key] = (w, U)
        d = phase_invariant_distance(U, target)
        best.append((float(d), w, U))

        if len(w) < max_len:
            for g in alphabet:
                if len(w) > 0 and inverse[g] == w[-1]:
                    continue
                q.append(w + [g])
```

关键在于： 1，BFS 保证短的 braid 先找到，每层扩展 4 种可能 1 ，2 ，1-， 2-，符合如下逻辑：先找最短长度的 braid，再找更长的。 2， 哈希剪枝的性能优化： 通过把 U 的数值矩阵 round 到 12 位 key=round(U) ，如果两个不同的 braid 产生几乎相同的矩阵，则只保留一个，从而大幅减少搜索树体积。 这样可以把搜索空间从 4^L 减少到约 3^L。

结果分析

demo流程是：

```
给定目标门 H（Hadamard）：

搜索所有长度 ≤ 8 的 braid

计算误差

返回误差最小的 6 个

打印结果

即实现了给定目标量子门 → 找出最优 braid 实现”
```

结果如下：

```
Searching for approximations up to braid length 8...

=== Best approximations for Hadamard (H) ===
distance=1.684162e-01  length= 3  word=1 2 1
distance=1.684162e-01  length= 3  word=1- 2- 1-
distance=2.948750e-01  length= 6  word=1- 2- 2- 2- 2- 1-
distance=2.948750e-01  length= 6  word=1 2 2 2 2 1
distance=2.948750e-01  length= 4  word=2 1- 1- 2
distance=2.948750e-01  length= 4  word=2- 1 1 2-

=== Best approximations for Pauli-X (X) ===
distance=1.833671e-01  length= 8  word=2- 2- 2- 2- 1 1 1 2-
distance=1.833671e-01  length= 8  word=2- 1 1 1 2- 2- 2- 2-
distance=1.833671e-01  length= 8  word=2 1- 1- 1- 2 2 2 2
distance=1.833671e-01  length= 8  word=2 2 2 2 1- 1- 1- 2
distance=3.362348e-01  length= 7  word=1 2 2 2 2 2 1
distance=3.362348e-01  length= 7  word=1- 2 2 2 2 2 1-
```

结果显示了在 braid 长度 ≤ 8 的范围内寻找最优近似 Hadamard、Pauli-X 的 Fibonacci 任意子编织序列

- `word=1 2 1` 表示 $\sigma_1\ \sigma_2\ \sigma_1$ 的组合
- `1-` 表示 σ_1^{-1}
- `distance` 是编织得到的门和目标门（如 Hadamard）之间的 SU(2) 距离
- `length` 是编织的长度

这个结果完全符合 **Freedman–Kitaev–Larsen–Wang** 的拓扑量子计算理论。

结果可以解决的实际问题：

1. 自动生成拓扑量子门

可以用于模拟：拓扑量子比特，任意子的编织线路...

2. 编织量子门的搜索

3. 构造拓扑量子算法， 例如Fibonacci 任意子的 Hadamard

4. 用于模拟非阿贝尔任意子的量子计算实验 ...